

Single-Pass Drone Video to a Complete 3D Model on One GPU:

Overlap-Band Keyframes, a Parallax Test for “Is It 3D?”, Structure from Optical Flow and Fused Depth

Team SIH_2D-to-3D

Smart India Hackathon 2026 — Problem Statement 26158 (NTRO)

Abstract

We present an end-to-end system that turns a single drone pass — one video, no flight planning, no ground control — into a textured 3D mesh and a dense point cloud and, when a flight log is available, georeferenced metric outputs in the formats problem statement 26158 asks for (OBJ, PLY, LAS, GeoTIFF, glTF, FBX), with a web viewer. Every stage runs on one GPU. A keyframe selector formalises the intuition that consecutive views must be *similar enough to match and different enough to triangulate*: it chains optical flow to measure co-visibility and picks the sharpest frame inside an overlap band $[\tau, \tau + \delta]$. Before any reconstruction it answers whether a pass contains recoverable 3D structure at all, from rotation-compensated parallax measured with *direct* keyframe-pair flow and normalised by an estimated noise floor. The same flow then replaces feature matching: tracks from direct RAFT flow give camera poses within 0.19% of the flight extent and 0.28° of SIFT structure from motion, triangulated flow against wide keyframe baselines gives dense depth, a monocular prior calibrated to it fills what flow cannot see, and a GPU TSDF fuses the views. A 187s video is processed in —s against a budget of —s (15 minutes per 10-minute video), covering — of the non-sky pixels its keyframes saw. On a straight single-pass track the usual 7-DoF GPS alignment silently rolls the model about the flight axis; levelling on the ground plane and fitting four degrees of freedom removes the failure, and with synthetic GPS on real models the exported mesh is within 0.43–2.06 m of the truth near the track. Overlapping what does not depend on each other — mapping one pass while the GPU tracks the next, writing files while it bakes the next texture — brings the 55s Jal Mahal edit to 76s against its 82s budget. A resident engine keeps the networks warm, loads a model only when the GPU has room for it, and reconstructs live streams segment by segment; a web console exposes every option and every layer of the result — mesh, dense cloud, Gaussian splats trained from that cloud, the keyframes and their fused depth — in one explorer. A slower profile trains depth-supervised 3D Gaussian Splatting. All numbers are produced by scripts in the repository from the run directories.

1 Introduction

Operational mapping — disaster response, inspection, reconnaissance — often has one chance to fly over a site. Classical photogrammetry assumes the opposite: planned grids, high overlap, several passes and ground control points. Problem statement 26158 asks for an accurate, textured, georeferenced 3D model from *a single pass* of a moving UAV [1].

A single pass is hard for three distinct reasons, and each one maps to a component of our system.

1. **Which frames?** A video carries thousands of highly redundant frames and a few bad ones (motion blur, compression, fades). Sampling at a fixed rate is the common default; it either wastes compute or, where the drone moves fast, leaves gaps that break the reconstruction into pieces (§3.3).
2. **Is it 3D at all?** A drone that yaws on the spot, or looks at a distant or planar scene, produces video in which a homography explains every frame pair. No amount of computation recovers depth from such footage; the system should say so before spending it (§3.4).
3. **Where is it?** A single pass is nearly a straight line. Aligning the model to GPS through camera centres alone leaves the roll about that line undetermined (§3.8).

A fourth constraint is time: the statement’s budget is 15 minutes for a 10-minute video, and it scores processing speed (20%) next to accuracy (30%) and completeness (20%). Around these we build two GPU pipelines. The *fast profile*, the deliverable, reuses the selector’s optical flow (RAFT [2]) for structure from motion (pycolmap’s global mapper [3]) and dense depth, completes it with Depth Anything V2 [4], and fuses a textured mesh and point cloud in a GPU TSDF [5, 6] (§3.10). The *accurate profile* uses spirula-studio’s SIFT structure from motion and 3D Gaussian Splatting [7, 8] with the Marigold v2 depth prior [9] aligned to, and scored against, the SfM points.

Contributions. (i) An overlap-band keyframe selector over chained dense flow with an explicit multiplicity model, and pass segmentation for edited footage. (ii) A

per-pass 3D verdict from direct-flow residual parallax with a data-driven noise floor, validated on a synthetic pure-rotation control; we report where Torr’s GRIC [10] fails on flow correspondences. (iii) A measurement of chained versus direct optical-flow correspondence error against exact ground truth. (iv) A ground-levelled 4-DoF georeferencing rule with a significance test for sloped terrain, leave-one-out error and a jackknife scale uncertainty. (v) Structure from motion and dense depth from the selector’s optical flow instead of features, with a measurement of the triangulation baselines a single pass offers, and a study of the TSDF truncation band as the lever on completeness. (vi) An engineering study of fitting the budget on one GPU: a single decode for analysis and keyframes, a motion-adaptive analysis rate, RAFT without host syncs under CUDA Graphs, GPU texture baking, overlapped mapping and export, and per-stage time and utilisation recorded against the budget. (vii) A system around it: a warm GPU engine that shares a card safely, live reconstruction from a stream in segments, Gaussian splats on the fast profile’s poses started from the dense cloud, and a console whose explorer shows every layer of a reconstruction.

2 Related work

Structure from motion and splatting. Incremental SfM as in COLMAP [11] remains the reference for camera recovery; spirula-studio re-implements an incremental mapper on the GPU and adds sequence-aware matching for video [7]. 3D Gaussian Splatting [8] represents a scene as anisotropic Gaussians optimised by differentiable rasterisation; gsplat [12] provides the rasteriser we use for fly-through rendering.

Keyframe selection. Visual SLAM systems insert keyframes by co-visibility and tracking quality, and ORB-SLAM chooses between homography and fundamental-matrix initialisation by model scores [13]; Torr’s GRIC formalises that choice [10]. We use dense flow with the forward-backward consistency test of Sundaram et al. [14].

Monocular depth. Marigold [15] repurposed a latent diffusion model for affine-invariant depth; Marigold v2 [9] fine-tunes the Qwen-Image-Edit diffusion transformer [16] to predict affine-invariant log depth in one step. Depth Anything V2 [4] is a strong discriminative alternative.

Compositional reconstruction. WorldSculpt [17] lifts per-object priors into cluttered scenes from posed, masked views; we discuss it as a downstream extension in §6.

3 Method

3.1 GPU ingest

Frames are decoded by NVDEC through `ffmpeg (-hwaccel cuda)`, resized on the GPU (`scale_cuda`)

and leave the decoder as raw NV12; conversion to RGB (BT.709, limited range) runs in PyTorch on the GPU. A background thread streams chunks so decoding overlaps optical flow. Keyframes are later re-decoded at full resolution, selected by timestamp in groups of 24 on two concurrent decoder sessions, cropped on the GPU and encoded with `nvJPEG`.

3.2 Pass segmentation

For analysis frames $I_0, \dots, I_{N-1}$ (long side 640 px, ≈ 12 fps) we compute RAFT flow $\mathbf{f}_t$ from t to $t+1$ and $\mathbf{b}_t$ back, and the share c_t of pixels passing the forward-backward test $\|\mathbf{f} + \mathbf{b}\|^2 < 0.01(\|\mathbf{f}\|^2 + \|\mathbf{b}\|^2) + 0.5$ [14]. Inside a continuous camera move c_t stays high; a cut, fade or dissolve collapses it. A *pass* is a maximal run with $c_t \geq 0.35$, no dark or flat frame, and at least 2s; its ends are trimmed while they are darker than $0.8\times$ the pass median, removing fade-ins and fade-outs (a fade kills tracks, so without trimming it attracts keyframes, and a fade frame is a meaningless held-out view). Letter-box bars are found before analysis from the per-row and per-column maximum luma over twelve seek-sampled frames and cropped on the GPU: static black would otherwise count as perfectly tracked content, inflating overlap, and as trivially correct pixels, inflating held-out PSNR. Real footage is often edited; a true single pass yields one pass, and SfM rejoins passes that turn out to share views.

3.3 Overlap-band keyframes

A grid G of points (8px spacing) on keyframe K is chained through the consecutive flows; a track dies when it fails the consistency test or leaves the frame. With $A_t \subseteq G$ the tracks alive at frame t ,

$$\text{ov}(K, t) = \min \left(\frac{|A_t|}{|G|}, \frac{|\mathcal{C}(A_t)|}{|\mathcal{C}|} \right), \quad (1)$$

where $\mathcal{C}$ are the 32px cells of frame t and $\mathcal{C}(A_t)$ those containing a live track. The first term falls as K ’s content leaves the view (forward motion), the second as new content enters it (backward motion), so the measure is symmetric in effect. The next keyframe is the sharpest frame (Laplacian variance relative to a 2s rolling median) among those with $\tau \leq \text{ov}(K, t) \leq \tau + \delta$ before the curve first drops below τ ; defaults $\tau = 0.75$, $\delta = 0.1$. If the overlap drops below τ within one step, the next frame is taken to keep the chain connected.

Multiplicity. Under uniform motion a point stays in view for about $1/(1 - \tau)$ keyframe spacings, so each surface point is observed by $m \approx 1/(1 - \tau)$ keyframes ($m = 4$ at $\tau = 0.75$) — the “two or more frames with a proper threshold of similarity” that triangulation needs, stated as a design parameter. We report the measured multiplicity, $1 + 2\bar{L}$ with $\bar{L}$ the mean forward track length in keyframes.

Cost. Only consecutive pairs are flowed, so analysis is linear in video length. An overlap matrix $M_{i,d} = \text{ov}(i, i+d)$ for all frames and $d \leq d_{\max}$ is built in one

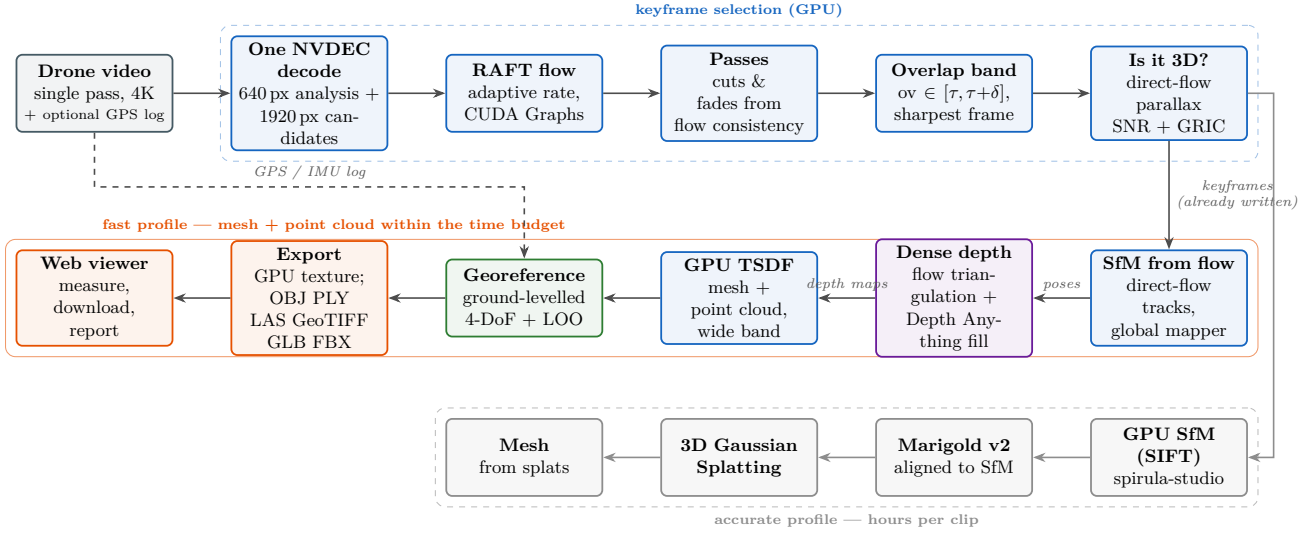


Figure 1: Pipeline. The keyframe selection is shared; the fast profile (the deliverable) reuses its optical flow for poses and depth and fuses a textured mesh and point cloud, the accurate profile trains Gaussian splats. Everything runs on one GPU; stages exchange files only, so any stage can be re-run alone.

sweep: a fresh grid starts at every frame and all live grids advance together in a ring of $d_{\max}$ slots, one batched `grid_sample` per frame and a single read-back; selection is then a CPU walk over M .

3.4 Is it 3D? A parallax test

For each keyframe i we take its widest partner $j \leq i + 6$ with $ov \geq 0.3$, compute RAFT flow *directly* between I_{K_i} and I_{K_j} in both directions, and keep consistent grid correspondences $(\mathbf{x}_n, \mathbf{y}_n)$. On a batch of all pairs we fit, by normalised DLT [18] with Cauchy IRLS, a homography H and a rank-2 fundamental matrix F . The residual $\|\mathbf{y} - H\mathbf{x}\|$ is rotation-compensated parallax: zero for pure rotation or a planar scene, growing with depth variation times baseline. We estimate the per-coordinate noise $\hat{\sigma} = 1.4826$ median $\sqrt{e_F^2}$ from F 's Sampson residuals e_F — pure noise whenever F is correct, which includes degenerate motion — and normalise

$$SNR = \frac{\text{median}_n \|\mathbf{y}_n - H\mathbf{x}_n\|}{\sqrt{2}\sqrt{2\ln 2} \hat{\sigma}}, \quad (2)$$

where the denominator is the median transfer residual that isotropic noise in both views leaves, so $SNR = 1$ means no measurable parallax. A pass is *3D* when its median SNR over keyframe pairs is at least 2 and GRIC [10] (both models scored on geometric error with $\hat{\sigma}$) prefers F on at least 60% of pairs; *degenerate* when the SNR is below 1.5 and its 90th percentile below 2.

Why direct flow. Chaining consecutive flows makes long-range correspondence cheap but its error accumulates with a systematic per-step bias (§5, Fig. 4); over a keyframe span the drift reaches the size of the parallax being measured and is correlated across points, which the extra freedom of F partly absorbs. Direct flow keeps the error near 0.1 px at the same gaps.

3.5 Structure from motion

Keyframes are written per pass (`images/pass_k/`) and reconstructed by spirula-studio's GPU SfM with each pass declared a sequence, so neighbours are matched and trusted first, plus GPU pair selection for loop closure. We use a *single-focal* radial camera per pass: drone cameras have square pixels, and with independent f_x, f_y the weak geometry of forward or orbital motion let the two drift 10–20% apart for no gain in reprojection error (§5). Separate passes of an edited video become separate models unless they share views.

3.6 Monocular depth prior

Marigold v2 [9] predicts, per keyframe, $p(\mathbf{u})$ with $\log d(\mathbf{u}) = ap(\mathbf{u}) + b$ for unknown a, b . It runs as upstream: 4-bit NF4 base transformer (what its LoRA was trained on), rank-128 LoRA and fine-tuned VAE decoder, one rectified-flow step, bf16 autocast, at 1024 px; the quantised base is cached so a load reads ≈ 11 GB rather than re-quantising 39 GB. For every registered image the SfM tie points give depths z_n at pixels $\mathbf{u}_n$.

Affine is not enough at landscape scale. A Cauchy-IRLS fit of $\log z_n = ap(\mathbf{u}_n) + b$ works well where the tie points span a narrow depth range, but on drone footage the prior *compresses the far field*: on Jal Mahal it places the far shore at the depth of the palace and the mountain ridge barely beyond it (Fig. 5), so a single slope cannot fit both. We therefore calibrate each image with a monotone map: medians of $\log z$ in quantile bins of p , made non-decreasing by weighted pool-adjacent-violators, linear with the affine slope outside the knots. It keeps the prediction's ordering and lets the mapping bend. Both calibrations are scored on *held-out* tie points (5-fold cross-validation per image).

Where the prior saturates. In a distant pass every tie point may sit where the prediction has saturated at its far end; the correlation between p and $\log z$ is then near zero or negative. The robust fit returns a non-positive slope and the stage writes no depth map for that image, so training is not supervised with wrong geometry.

The calibrated linear depth supervises splat training through spirula’s Pearson-correlation depth loss (weight 0.05). Because that loss is affine-invariant in *linear* depth, the calibration — which only SfM can provide — shapes the supervision itself, not merely its reporting. *What* is supervised matters as much: outside the range of predictions the tie points cover (sky, featureless far field) the calibration only extrapolates, and a depth invented for the sky asks the trainer to put splats there. The mask therefore keeps pixels whose prediction lies within the 0.5–99.5% range of the tie-point predictions (plus 5% margin) and below $3\times$ the 99th-percentile SfM depth. The canonical run below was trained before this mask existed (only the far cut, which removed nothing); the masked maps are evaluated as an ablation (§5).

3.7 Gaussian splatting and meshing

Each SfM model with at least 8 images trains a 3DGS model in spirula-studio (30k steps, half resolution, bilateral-grid exposure correction), with every 8th keyframe held out for evaluation; meshes are extracted from the trained splats with vertex colour and texture atlases.

3.8 Georeferencing a straight pass

With GPS positions $\mathbf{g}_n$ (local ENU) for camera centres $\mathbf{c}_n$, the usual fit is the 7-DoF similarity of Umeyama [19]. If the $\mathbf{c}_n$ are nearly collinear the rotation about the line is unobservable and GPS noise sets it; the in-sample RMSE still looks like GPS noise because the unobservable direction does not affect it. We instead estimate the model’s ground normal $\mathbf{n}$ by RANSAC on the sparse points (oriented towards the cameras), rotate $\mathbf{n}$ to $+z$, and fit yaw, scale and translation — scale and yaw by complex least squares on horizontal coordinates, the vertical offset as a mean residual. The full similarity is used only if the track spans two dimensions *and* its tilt against $\mathbf{n}$ exceeds 5° by three jackknife standard errors, i.e. GPS proves the ground is sloped. We report leave-one-out RMSE rather than in-sample RMSE, and a jackknife relative standard error of the scale as the scale check.

3.9 Keeping the GPU busy

Stock torchvision RAFT rebuilds its correlation lookup offsets on the CPU in each of its 12 refinement iterations, divides by a CPU tensor, builds coordinate grids on the CPU, and upsamples the flow after every iteration although inference keeps only the last; at 640 px a call cost roughly the same whatever the batch size, a sign of launch- and sync-bound execution. Removing these (identical arithmetic, differences $\leq$ bf16 noise)

and replaying the network from a CUDA Graph raised throughput 1.3-fold. The selector’s tracking avoids boolean-mask indexing, whose `nonzero` forces a host sync, in favour of scatter and redirected writes. Every stage runs under an NVML sampler recording utilisation, power, memory and the presence of other processes on the GPU, so no timing is reported without its context.

3.10 A fast profile for the time budget

The accurate profile spends hours per clip on splat training and meshing; the problem statement asks for a mesh or point cloud within $1.5\times$ the video length. The fast profile keeps the selector and replaces everything after it.

One decode. The analysis decode also delivers every analysis frame at the output size: ffmpeg splits the NVDEC stream, scales both branches with `scale_cuda` and stacks them in one NV12 frame, so one pipe carries both and cannot deadlock; the planes are split before colour conversion so chroma does not bleed across the seam, and candidates are nvJPEG-encoded as they stream. The chosen keyframes are then files already written, where a second, seek-heavy 4K pass took 31 s for a 55 s clip. Twelve frame pairs one nominal analysis step apart, sampled across the video, give the median flow per step; the analysis stride grows until a step moves about 2 % of the width (at least 3 analysis frames per second). The sample videos move 0.5–3 px per step at 12 fps.

Structure from flow. For keyframes $K_1, \dots, K_n$ of a pass at 960 px we compute direct RAFT flow from each K_i to $K_{i+1}, \dots, K_{i+3}$ in both directions. Points seeded on a grid where no track is alive move from K_i to K_{i+1} by sampling the flow at their sub-pixel position (forward–backward consistent only); each step is also predicted by the direct flow from K_{i-1} and K_{i-2} , and a track whose chained position disagrees with a prediction by more than 1 px ends, while agreeing predictions are averaged, so chained drift (§3.4) cannot accumulate. The tracks are written as COLMAP keypoints at full resolution with matches between keyframes up to four apart, verified by RANSAC and mapped by the global mapper [3], capped at about twelve tracks per image. The incremental mapper’s defaults — a 16° initialisation angle, 1.5° to triangulate — reject whole single passes, whose nearby keyframes are $0.1\text{--}1^\circ$ apart at the scene.

Depth from flow, completed by a prior. With the poses fixed, every consistent flow vector from a reference keyframe to neighbours $d \in \{2, 4, 8, 12\}$ keyframes away is triangulated by the midpoint of the two rays; a depth is kept if the rays meet at $\geq 0.5^\circ$ and it reprojects within 1.5 px in both views. Consecutive keyframes are too close for this — their median triangulation angle on Jal Mahal is $0.04\text{--}0.4^\circ$, twelve apart $0.6\text{--}1^\circ$ — hence the wide gaps. Candidates from all neighbours are fused per pixel (median, then an angle²-weighted mean of

those within 5 %, at least two views). What flow cannot triangulate (water, far field, low texture) is filled by Depth Anything V2 [4], calibrated per image to that image’s triangulated pixels with the monotone map of §5; its sky is exactly zero disparity, and those pixels are never filled.

Fusion, texture and export. All depth maps are integrated in an Open3D GPU TSDF [5, 6] whose voxel is three pixel footprints at the median depth and whose truncation band is twelve voxels (§5.7). The mesh is textured from the 1920 px keyframes on the GPU without UV unwrapping: triangles are packed two per square cell of a 4096^2 atlas, each triangle takes the keyframe that sees it most frontally and closely without occlusion (a z-buffer of triangle centroids per view), and each texel samples that keyframe at its point on the triangle. Exports are PLY (full density), GLB, textured OBJ/GLB and FBX (a copy with at most 600k triangles), LAS, and a DSM and orthophoto rasterised from the point cloud; with georeferencing, LAS and GeoTIFF are in UTM with the EPSG code (ENU $\rightarrow$ ECEF $\rightarrow$ UTM, not ENU offsets). A self-contained three.js viewer opens at the middle keyframe’s viewpoint and measures distances and height differences.

Staying inside Open3D. Open3D’s GPU mesh extraction fails once a grid holds more than about 65k active blocks — 62,760 worked, 66,472 did not, at any hash-map capacity, and the CPU path segfaults — with an “unspecified launch failure” that poisons the CUDA context for the rest of the process. The voxel therefore grows until the surface estimate stays under 60k blocks, the fused grid is checked and re-fused once if it is still over, and the hash map is held to 100k blocks, the largest size tested safe for several grids in one process; at the quality profile’s resolution this changed completeness by under 0.01.

Sampling once. Before the analysis decode, the stage looked for letterbox bars in twelve frames and probed the motion in twelve frame pairs, one ffmpeg process per sample. Creating a decoder context per process cost more than the decode itself, and each seek decoded from the previous key frame (the clips’ GOPs are 5.3 s): 11.9 s of Jal Mahal’s run. The samples now start on key frames read from the packet index, are decoded on the CPU, and one set serves both tests: 2.2 s. The viewer’s decimated copy is made with a C++ quadric decimator (8.4 s instead of Open3D’s 56 s for a 3.7M-triangle model, as close to the surface).

Time. Every stage’s wall time is recorded next to the budget. Two host-side traps cost more than any kernel: the login shell’s `OMP_PROC_BIND=spread` made the first OpenMP runtime pin each process, and every thread and child it later started (the mapper, ffmpeg), to core 0 — the global mapper got 0.13 cores until re-pinned — and torch’s CPU path spent 48 s calibrating 97 depth maps with 24 threads per small operation, 3 s in numpy.

4 System: a warm engine, live footage and a console

A warm engine. `drone3d engine` is a resident process per GPU (Fig. 2). Every stage takes its networks from a process-wide cache: outside the engine nothing changes — each network is built and freed per use — while inside it RAFT, Depth Anything and, on request, Marigold stay loaded, one network serves every batch size, and RAFT’s CUDA graphs for a resolution are captured once. Runs write the same files as the command line. On Jal Mahal a warm run took 106.3 s against 109 s cold under the same conditions: warmth saves seconds, not minutes; its point is an engine that is always ready and live segments that start at once.

Sharing a GPU safely. A mapping server is rarely a dedicated machine, so the engine is built to run beside other GPU work. A model is loaded only if NVML shows its measured footprint plus a reserve free at that moment, other processes included; it is never unloaded while a run uses it (the request waits for the run); and before each run the memory-hungry settings — TSDF budget, flow batch, texture size — are fitted to the memory then free. Open3D and CUDA faults are sticky: after every job the engine checks that its context still works, and if not it saves its queue and warm list, leaves a marker and exits, and the console starts a fresh engine that re-warms the same models and resumes the queue.

Live footage. The problem is posed for recorded video, but a model of the first minutes of a flight is useful while the drone is still flying. ffmpeg reads the link (RTSP, RTMP, SRT, UDP, HLS, HTTP, or a V4L2 camera encoded on NVENC) and cuts it at keyframes into fixed-length segments without re-encoding; each closed segment becomes a run on the engine, ahead of other work. With a flight log every segment is georeferenced into one ENU frame, its origin pinned to the log’s first fix, so that the segments’ models line up in one scene. A recorded flight can be replayed at its own frame rate to rehearse the whole path (§5.12).

The console. `drone3d ui` serves a web console. It shows the engine and each model it holds, with load and unload controls; NVML gauges and ten-minute charts, with our processes told apart from other users’; a check of what the machine can do (CUDA, NVDEC and NVENC, nvJPEG, Open3D on the GPU, the global mapper, the splat trainer, the FBX and Blender writers, the weights); and work in flight against the time budget. A run is configured from a video or a stream, a profile, a resolution preset and module switches, and every option of the configuration is available, generated from the configuration’s data classes and the comments beside each field so that the form cannot drift from the code. The explorer (also shipped with every export) shows each layer of a reconstruction as a switch: the textured mesh (texture, shading or wireframe), the dense cloud, the Gaussian splats, the flight path with its frusta, the keyframe photos and their fused depth maps as image

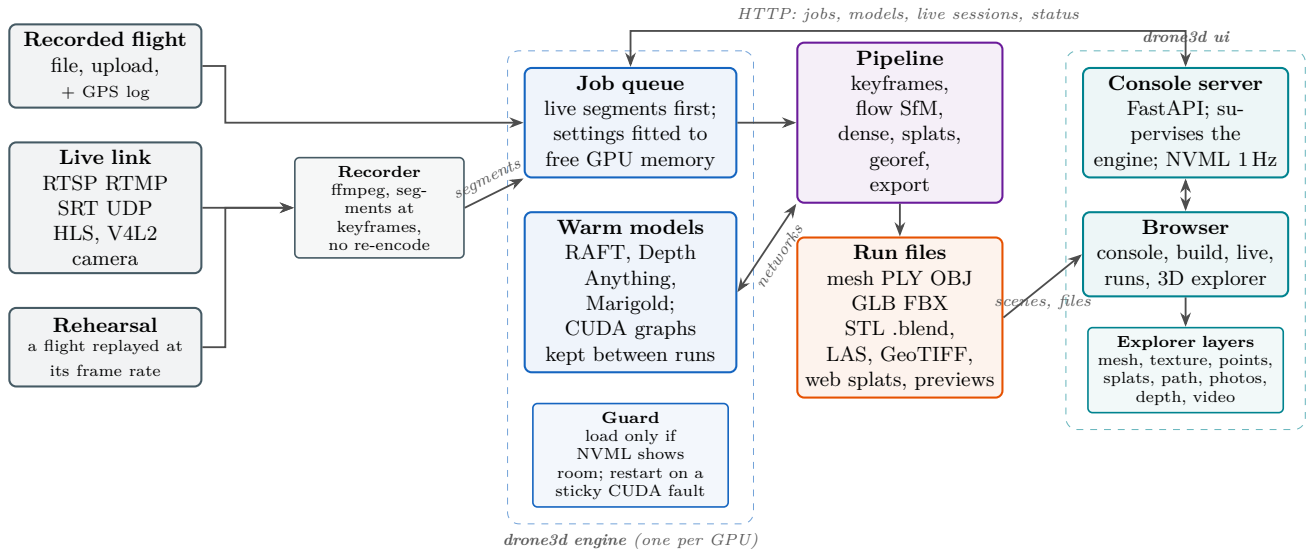


Figure 2: The system around the pipeline. The engine keeps the networks on the GPU between runs and executes jobs — recorded flights or live segments — one at a time; the console supervises it, samples the GPU and serves every layer of a reconstruction to the explorer.

planes where the drone took them, and the source video, which follows the camera when the explorer replays the drone’s own path. Clicking a keyframe puts the camera where the drone was, with its field of view, and lays the photo over the model. Deliverables per model are PLY, OBJ, GLB (y-up, as glTF requires), FBX, STL, a Blender scene with its texture packed, LAS and GeoTIFF, and the web splats.

Keeping the engine alive. A resident process meets failures a one-video process never does. After a dozen videos in one engine, Open3D’s GPU mesh extraction hit an illegal memory access on a grid well inside every limit of \$3.10 — state accumulated across grids, not the grid itself — and a sticky CUDA error poisons every later call in the process. Inside the engine the dense stage therefore fuses in a spawned child process, started with the stage so that it boots while depth is computed: a fault kills the child, the model is retried in a fresh one, and the engine’s own context and warm networks are untouched; results are identical (Jal Mahal: same completeness and triangle counts) for about 2 s plus 0.5 s per model. If the engine’s context is nevertheless lost, it saves its queue and warm list and exits, and the console restarts it with the same models warm; we observed exactly this once during the benchmark, recovered in about a second, with the next job at the front of the restored queue. New code or a new slot count is applied the same way, by draining: the engine takes no new job, lets running ones finish, and hands its queue to its successor. Two further lessons came from checking capabilities for real: ffmpeg lists NVENC whenever it was built with it, but the A100 has no encoder engine, so a one-frame encode decides and live cameras fall back to x264; and a busy engine can take seconds to answer HTTP (its interpreter lock is shared with the run), so the console judges it by its live process rather than by a short timeout.

Testing. The repository’s tests (pytest, 241 at the time of writing) cover every stage’s arithmetic on synthetic data with known answers, the formats’ round trips (y-up GLB, STL, LAS with its CRS, GeoTIFF geokeys, the web splat layout and its frame change), the engine’s queue, slots, admission, drain and fault paths, the fusion worker’s recovery, and the console’s API, uploads included. An end-to-end check drives the console in GPU-accelerated headless Chrome: every page, every explorer layer (the canvas must change when it is switched), zoom, orbit, fly, follow-flight, measuring, screenshots, the source video, look-through-keyframe and the depth comparison, and a request for every deliverable; it fails on any console error or failed request. It found three real faults the unit tests had not: uploads failing on an unresolved type annotation, the thumbnail cache listed as keyframes, and textured meshes rendered black after a layer switch.

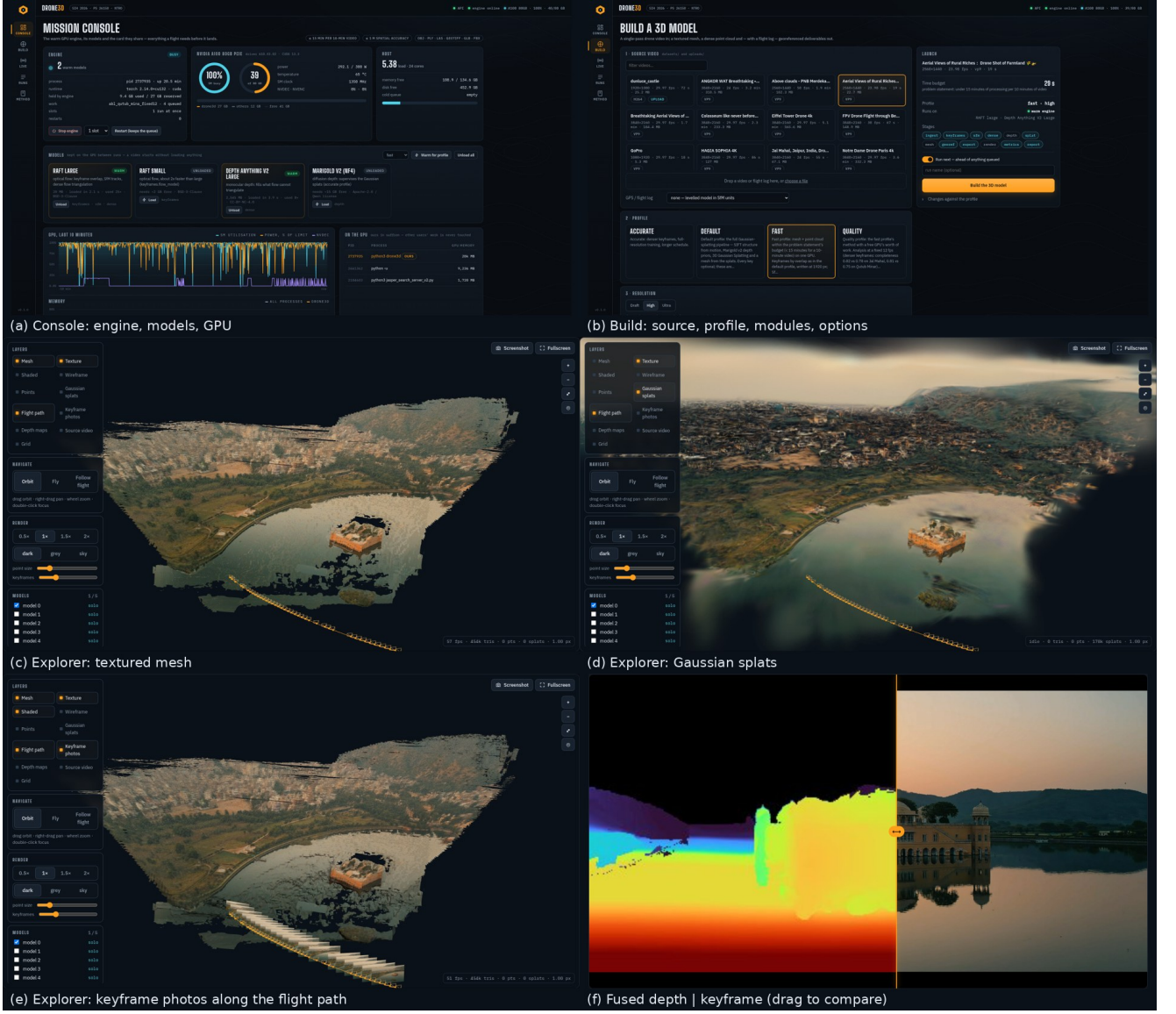


Figure 3: The console, rendered headless on the GPU. (a) The warm engine, the models it holds and the GPU; (b) building a model from an uploaded video, every option available; (c–e) the explorer with the textured mesh, the Gaussian splats trained from the dense cloud, and the keyframe photos placed where the drone took them; (f) a keyframe against its fused depth.

5 Experiments

Setup. One NVIDIA A100 80 GB PCIe and 24 CPU cores; the fast profile’s timings are taken on the warm engine running one job at a time. PyTorch 2.14.0+cu132 with CUDA 13.2, Python 3.14, ffmpeg 9.0 (NVDEC/NVENC), spirula-studio 2026.9.24 (Vulkan backend). Data: fifteen public drone videos (Table 1); they carry no flight logs.

5.1 Passes, keyframes and the 3D verdict

On Jal Mahal the flow signal splits the edit into 5 passes at its cuts and fades, and the band selects 213 keyframes; the mean overlap with the previous keyframe lies in 0.74–0.86, inside or just above the target band, and every pass is judged *3d* (Table 2). Decoding and flow for the 658 analysis frames took 37 s, selection 12 s

Table 1: Sample footage (public YouTube re-encodes, VP9).

Video	Resolution	fps	s
ANGKOR WAT Breathtaking Cinemati	3840×2160	24.00	193
Above clouds	2560×1440	50.00	115
Aerial Views of Rural Riches	2560×1440	23.98	19
Breathtaking Aerial Views of Hanoi	3840×2160	29.97	103
Colosseum like never before	3840×2160	29.97	135
Eiffel Tower Drone 4k	3840×2160	29.97	306
FPV Drone Flight through Beautiful	3840×2160	30.00	67
GoPro	1080×1920	29.97	18
HAGIA SOPHIA 4K	3840×2160	29.97	85
Jal Mahal, Jaipur, India, Drone Ci	3840×2160	24.00	55
Notre Dame Drone Paris 4k	3840×2160	29.97	214
Petronas Twin Tower in FPV drone	3840×2160	59.94	35
Qutub Minar, New Delhi, India 4K D	3840×2160	29.97	187
Reichstag Berlin in 4K	3840×2160	30.00	130
The Messiah	3840×2160	29.97	94

Table 2: Pass segmentation and keyframes per run. Overlap: mean co-visibility with the previous keyframe; views: measured views per point; SNR: median direct-flow parallax SNR (1 = none).

Run	Pass	Time (s)	KF	Overlap	Views	SNR	Verdict
jal_mahal	0	0.0–7.4	30	0.77	6.9	22.4	3d
jal_mahal	1	8.0–14.1	14	0.79	5.2	3.6	3d
jal_mahal	2	14.2–23.8	59	0.77	7.7	6.0	3d
jal_mahal	3	23.8–36.0	97	0.74	7.4	16.3	3d
jal_mahal	4	37.2–54.0	13	0.86	8.2	3.2	3d

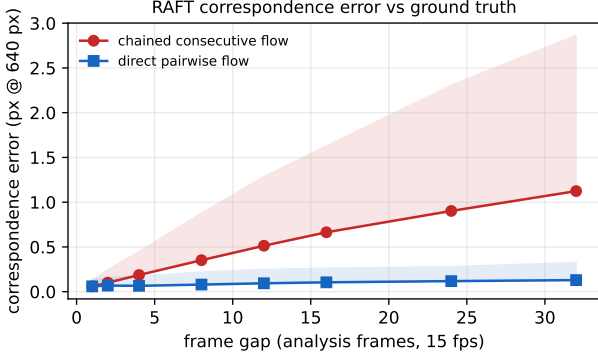


Figure 4: Correspondence error against exact ground truth on the rotation control. Chained consecutive flow drifts almost linearly (1.12 px median at 32 frames); direct pairwise flow stays at 0.13 px.

and 4K extraction 45 s.

Negative control. A synthetic clip of a camera rotating about its centre over one real 4K frame (`tools/make_degenerate_clip.py`) has no parallax by construction. It yields SNR 1.24 (median residual 0.02°) and the verdict *degenerate*; GRIC nonetheless preferred F on 60% of its pairs, which is why GRIC only confirms the SNR decision.

5.2 Structure from motion

With the same SfM settings, 1 fps sampling registers 40 of 55 frames in 2 disconnected models (the largest 22), and uniform sampling at the selector’s budget of 241 frames registers 237 but splits into 4 models (largest 98). The overlap band registers all 241 (largest 155); after cropping the letterbox and trimming fades the canonical run registers 213 of 213 (Table 4). A model per group of passes is expected: the passes of a cinematic edit are cut from different flights and need not overlap. One focal length per pass keeps the reprojection error while OpenCV’s independent f_x, f_y drift apart (Table 5).

Table 3: Structure from motion on the selected keyframes.

Run	KF	Registered	Models	Reproj. (px)	Time (s)
jal_mahal	213	213	4	0.79	255

Table 4: Frame sampling on Jal Mahal (same SfM settings).

Sampling	Frames	Registered	Models	Largest
1 fps (uniform)	55	40	2	22
uniform, same budget	241	237	4	98
overlap band (dev)	241	241	4	155
overlap band (canonical)	213	213	4	156

Table 5: Camera model on Jal Mahal: independent f_x, f_y (OpenCV) against one focal (radial).

Camera	Focal(s) per pass (px)	Reproj. (px)	Points
OpenCV (f_x, f_y free)	3112/2812, 2055/2080, 2827/3382, 2986/2754	0.761	96,940
radial (one focal)	3118, 2684, 2808, 3109	0.771	96,598

5.3 Monocular depth prior

Table 6 scores each calibration on tie points it was not fitted to. The monotone map lowers held-out AbsRel on every model with depth; the gain is largest where the scene spans a wide depth range (Fig. 5), which a single log-affine map cannot follow. Model 3 (30 keyframes, the far lake-side pass) has no image whose prediction correlates positively with its tie-point depths; it is trained without depth rather than with a wrong one.

Table 6: Marigold v2 against SfM tie points, scored on held-out tie points (5-fold cross-validation per image, median over images): log-affine vs monotone calibration.

Run	Model	Aligned	AbsRel aff.	AbsRel mono.	δ_1 aff.	δ_1 mono.
jal_mahal	0	156/156	0.050	0.027	0.857	0.886
jal_mahal	1	14/14	0.009	0.006	1.000	1.000
jal_mahal	2	13/13	0.075	0.033	0.945	0.989
jal_mahal	3	0/30	—	—	—	—

5.4 Gaussian splatting

5.5 Georeferencing a straight pass

None of the sample videos carries a flight log, so georeferencing is evaluated in simulation (`experiments/georef_study.py`): a 300 m track at 80 m altitude over flat ground with buildings, the SfM model hidden behind a random similarity, horizontal GPS noise, and a sideways deviation of the track from a straight line of 0–60 m; the error is measured at ground points 100 m off the track. (Sloped ground, where the rule falls back to the similarity, is covered by the unit tests.) The levelled fit is never worse than the similarity (worst 2.1 m) and its metric scale error stays below 0.6 % (Table 8, Fig. 6).

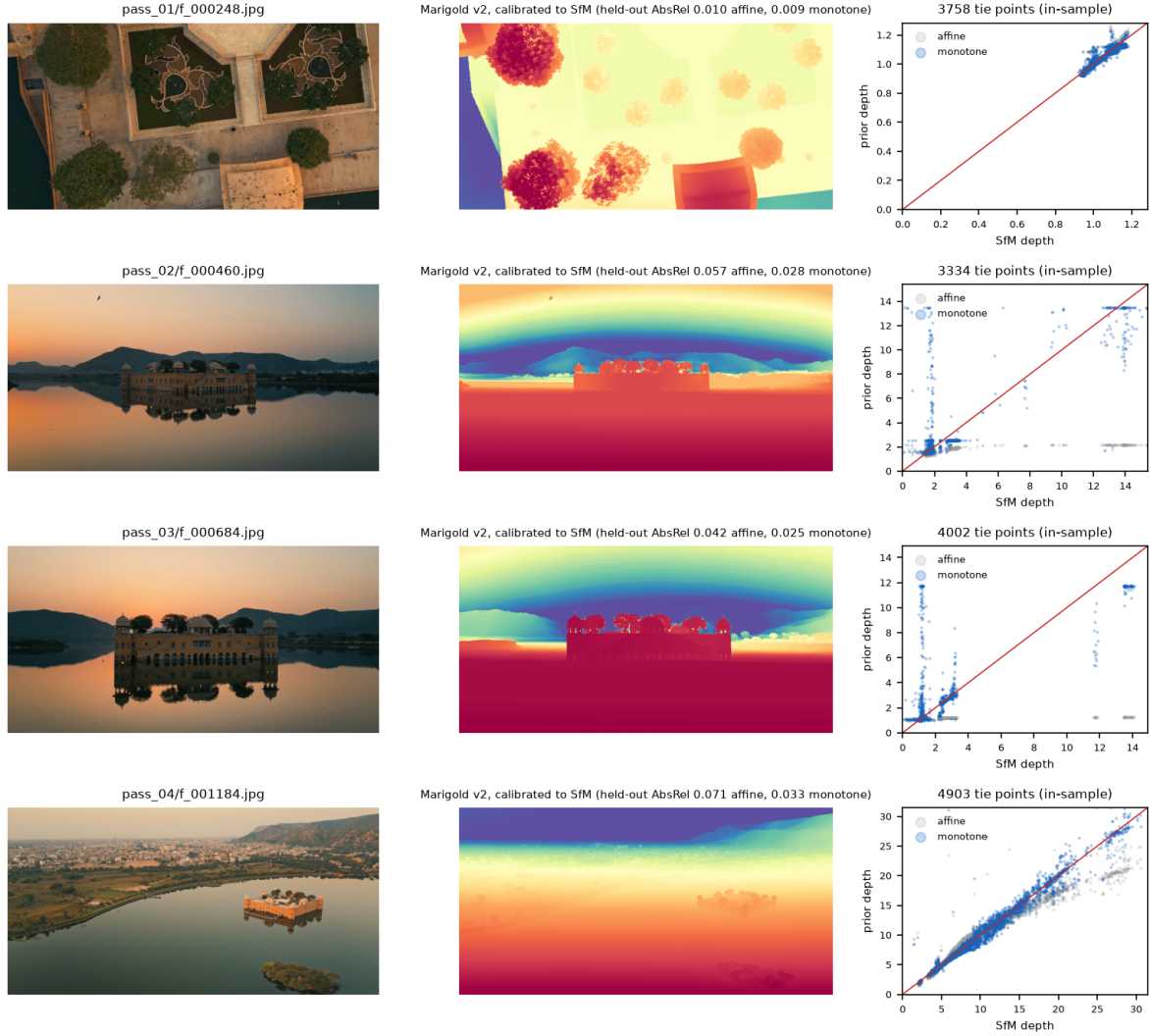


Figure 5: Marigold v2 on Jal Mahal keyframes (canonical run). Right: prior depth against SfM tie-point depth for the log-affine fit (grey) and the monotone calibration (blue). On the lake-facing passes the affine fit flattens the far shore and the ridge; the monotone map follows them (it also sends a few near points far). Held-out AbsRel for both is given above each depth map.

5.6 Throughput and GPU use

5.7 Fast profile: mesh and point cloud within the time budget

Structure from flow. On the 97-keyframe orbit of Jal Mahal, tracks from direct RAFT flow register every keyframe; with 960 px tracks the global mapper’s camera centres agree with spirula’s SIFT reconstruction to 0.19% of the flight extent and its rotations to 0.28° (median), with no feature extraction or matching (Table 11). Tracking at 640 px is cheaper but $2.5\text{--}3\times$ less accurate. COLMAP’s incremental mapper needs its initialisation angle lowered from 16° to 2° to start on single-pass footage at all and is eight times slower here.

Fusion. The depth maps (triangulated, then filled by the calibrated prior) cover every non-sky pixel of the references, yet a TSDF with the usual 4-voxel truncation band keeps only part of the view: neighbouring views disagree slightly and their signed distances can-

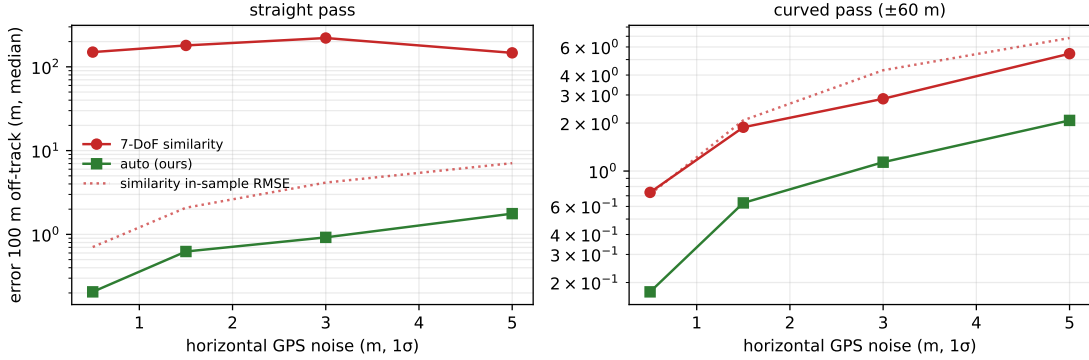
cel. Widening the band recovers most of it at a small cost in depth error against the triangulated geometry (Table 12); the profile uses 12 voxels.

Analysis rate and overlap. A fixed 12 fps analysis rate places more keyframes and gains a few points of completeness at a large cost in time; measuring overlap against the points that survive the first tracking step (so that water does not force a keyframe every step) does not pay off either (Table 13). The profile uses the motion-adaptive rate with absolute overlap.

Every sample video. On the warm engine the mesh-and-cloud stages processed 1 of the 1 sample videos within the budget (1 minutes of footage at 1.43s per second of video overall; Table 14). Every model was then given Gaussian splats, trained from its dense cloud; their time is reported separately, since the problem statement asks for a mesh or a point cloud.

Table 7: 3DGS on held-out keyframes (every 8th keyframe never used in training).

Run	Model	Splats	PSNR	SSIM	LPIPS	Train (s)
jal_mahal	model_0, depth prior (v1 mask)	3,000,000	35.00	0.977	—	3822

**Figure 6:** Error of ground 100 m off-track after GPS alignment (median of 20 synthetic scenes per point). The 7-DoF similarity is off by up to 221 m on a straight pass while its in-sample RMSE (dotted) only reflects GPS noise; the ground-levelled fit stays near the GPS noise.

Georeferencing, end to end. Each Jal Mahal model receives a known model-to-world similarity at the real site and its keyframes GPS with 1.5 m horizontal and 3.0 m vertical noise; the pipeline’s georef and export stages then run unchanged. The levelled fit recovers rotation to a few tenths of a degree and scale to half a percent; the exported mesh is within 0.43–2.06 m (median) of the truth within 100 m of the track, and the error grows with distance, as yaw and scale error do (Table 15). LAS and GeoTIFF outputs carry EPSG:32643 (UTM 43N).

5.8 Doing in parallel what does not depend on each other

The passes of a video are independent. The global mapper releases Python’s interpreter lock, so each pass is mapped on a thread while the GPU tracks the next, largest pass first so that its long mapping overlaps the tracking of all the others: SfM on Jal Mahal fell from 47.5 s to 22.5 s with the same 128/128 keyframes registered. Export writes the textured GLB/OBJ, FBX and Blender files in the background while the GPU bakes the next model, and no longer decimates meshes within 25 % of the viewer cap (16.5 s → 10.5 s). The per-stage GPU telemetry thread had been finding the run’s child processes by reading every `/proc/*/stat` twice a second, holding the interpreter lock against the stage it measured; it now walks `/proc/<pid>/task/*/children`. Together the 55 s Jal Mahal edit went from 109 s to 76 s, inside its 82 s budget for the first time (Table 16).

5.9 Gaussian splats on the fast profile’s poses

Flow SfM keeps about twelve tracks per image, too few points for 3DGS to densify from in a few thousand steps. Starting instead from the dense TSDF cloud — a copy of the SfM model whose points are the fused surface

— raises held-out PSNR from 27.5 to 29.0 dB in 35 s of training (Table 17). The trained splats are converted to the 32-byte web layout (importance-sorted, opacity-pruned) in the model’s export frame, the trainer’s own scene transform undone: 743 MB of PLY become 20 MB that a browser renders together with the mesh.

5.10 Exposure across keyframes

A drone’s auto-exposure drifts along a pass, and the atlas blends the best views of every triangle, so keyframes of different brightness leave blotches. Before baking, each view gets a gain per colour channel from the objective of OpenCV’s stitching gain compensator, written over triangles instead of panorama pixels: with N_{ij} triangles seen by views i and j and I_{ij} view i ’s mean colour over them, minimise $\sum N_{ij} [(g_i I_{ij} - g_j I_{ji})^2 / \sigma_N^2 + (1 - g_i)^2 / \sigma_g^2]$ with OpenCV’s $\sigma_N = 10$, $\sigma_g = 0.1$. It removes 23–45 % of the colour disagreement between overlapping views on Jal Mahal’s models, with gains within 10 % of one (Table 18).

5.11 Edge-aware depth refinement: a negative result

Filtering each fused inverse-depth map under its keyframe’s colours before fusion — OpenCV’s guided filter, or its Fast Global Smoother with the triangulated pixels trusted more than the monocular fill — was expected to sharpen boundaries. It does not pay: the TSDF already averages view noise, and on the lake model the filters smear the fill across depth discontinuities, which the fusion then rejects (Table 19). The option stays in the configuration (`dense.refine`), off.

5.12 Live footage

Qutub Minar was replayed at its frame rate into 30 s segments: of 7 segments, 4 held parallax and were

Table 8: Georeferencing study (medians over 20 trials): off-track error of the similarity and of our rule, in-sample RMSE of the similarity, held-out RMSE of ours, metric scale error.

GPS σ (m)	Lateral (m)	Sim. (m)	Ours (m)	Sim. in-sample	Ours LOO	Scale err.
0.5	0	149.9	0.21	0.71	0.73	0.1%
0.5	5	5.5	0.19	0.69	0.71	0.0%
0.5	20	2.1	0.19	0.68	0.70	0.0%
0.5	60	0.7	0.17	0.72	0.74	0.1%
1.5	0	180.0	0.62	2.08	2.16	0.1%
1.5	5	19.5	0.54	2.10	2.18	0.1%
1.5	20	5.0	0.58	2.16	2.24	0.2%
1.5	60	1.9	0.63	2.08	2.16	0.2%
3.0	0	220.7	0.92	4.15	4.29	0.2%
3.0	5	27.0	1.11	4.00	4.15	0.2%
3.0	20	7.6	1.29	4.28	4.44	0.3%
3.0	60	2.8	1.14	4.29	4.44	0.2%
5.0	0	146.9	1.77	7.08	7.32	0.3%
5.0	5	24.4	2.13	7.13	7.37	0.5%
5.0	20	11.0	1.59	6.92	7.14	0.4%
5.0	60	5.4	2.08	6.83	7.07	0.6%

Table 9: Micro-benchmarks. The last column is RAFT’s mean difference to stock torchvision in pixels.

Component	Rate	Unit	Δ (px)
RAFT stock (batch 16)	87	flows/s	—
RAFT, no host syncs (32)	109	flows/s	0.0000
RAFT + CUDA Graph (32)	110	flows/s	0.0000
4K VP9, ffmpeg 4.4 CUVID	94	fps	—
4K VP9, ffmpeg 9 NVDEC	164	fps	—
4K VP9, ffmpeg 9 CPU (16 thr.)	112	fps	—
nvJPEG 4K encode	1.3	ms	—

modelled 26–38s after they closed; the others were refused by the parallax test, as a camera that only turns should be.

Table 10: Per-stage wall time and GPU use (NVML, 0.5 s samples). Own/host: peak GPU memory and host RSS of our process tree.

Run	Stage	s	Util.%	W	GPU (GB)	Host (GB)
jal_mahal	keyframes	109	83	256	10.6	6.3
jal_mahal	sfm	256	83	213	9.9	4.2
jal_mahal	depth	522	84	265	24.3	11.2

Table 11: SfM from flow tracks on Jal Mahal pass 3 against SIFT SfM of the same keyframes (similarity-aligned).

Tracks	Mapper	Flow (s)	Map (s)	Reg.	Reproj. (px)	Centre (% ext.)	Rot. med. (°)	Rot. max (°)
640 px, 8 iterations	global	8.8	9.8	97/97	0.32	0.53	0.79	1.44
640 px, 12 iterations	global	13.1	18.7	97/97	0.32	0.55	0.66	1.35
960 px, 12 iterations	global	28.9	16.2	97/97	0.32	0.19	0.28	0.41
960 px, 12 iterations	incremental	28.9	129.1	97/97	0.35	0.40	0.42	1.59

Reference: spirula-studio SIFT SfM of all 213 Jal Mahal keyframes, 255 s for extraction, matching and mapping.

Table 12: TSDF truncation band on Jal Mahal’s two largest models (voxel = 3 pixel footprints at the median depth): view completeness, median depth error against the triangulated depth, triangles.

Model	Band (vox.)	Completeness	Depth err. (%)	Triangles
0	4	0.468	0.42	41,072
0	8	0.722	0.51	184,092
0	12	0.800	0.60	402,964
0	16	0.853	0.68	666,407
1	4	0.750	0.92	178,556
1	8	0.874	1.05	416,504
1	12	0.912	1.09	604,939
1	16	0.928	1.10	743,658

Table 13: Fast profile: analysis rate and overlap measure, view completeness and time against the budget ($1.5\times$ video length).

Video	Analysis	Overlap	KF	Reg.	Compl.	Time (s)	Budget (s)
Jal Mahal	adaptive, 3.0 fps	absolute	128	128	0.66	79	82 ✓

Table 14: Every sample video, warm engine, one job at a time: length, height, keyframes, registration, models (one per pass), view completeness, mesh-and-cloud time against the budget and per second of video, then splat training for every model and its mean held-out PSNR.

Video	s	Res.	KF	Reg.	Mod.	Compl.	Time (s)	Budget (s)	s/s	Splat (s)	PSNR
Jal Mahal	55	2160p	128	128/128	5	0.66	79	82 ✓	1.43	205	26.6

Table 15: Georeferencing with synthetic GPS on the real models: rotation error, scale error, held-out horizontal RMSE of the cameras, and median mesh error by distance from the flight track.

Model	Fit	Rot. ($^{\circ}$)	Scale (%)	LOO (m)	<100 m	100–300 m	>300 m
0	yaw-scale	0.01	0.35	1.93	2.06	2.46	7.3
1	yaw-scale	0.23	0.07	1.99	0.86	1.19	8.7
2	yaw-scale	0.36	0.44	1.86	—	2.53	68.8
3	yaw-scale	0.02	0.47	2.11	0.43	0.72	—
4	yaw-scale	0.34	0.50	2.74	—	—	17.8
5	yaw-scale	0.06	0.50	1.97	—	—	—

Table 16: Jal Mahal, fast profile, idle A100, same conditions: seconds per stage before and after overlapping independent work.

Stage	Before (s)	After (s)
keyframes	27.1	26.9
sfm	47.6	22.5
dense	17.7	16.0
export	16.5	10.5
total	108.9	75.9 ✓

Table 17: Splat initialisation on Jal Mahal’s largest model (41 keyframes), 7000 steps, held-out views: splats, training time, PSNR, colour-corrected PSNR, SSIM.

Start	Quality	Splats	Train (s)	PSNR	cc-PSNR	SSIM
SfM points (1000)	high	20,000	22.2	27.46	31.06	0.886
dense TSDF cloud (400k)	low	200,000	34.6	29.01	35.59	0.948
dense TSDF cloud (400k)	medium	1,000,000	75.4	29.79	36.47	0.951
dense TSDF cloud (400k)	high	3,000,000	122.7	29.47	36.38	0.950

Table 18: Gain compensation on Jal Mahal: overlapping view pairs, mean colour disagreement between them (8-bit levels) before and after, and the range of the gains.

Model	Pairs	Before	After	Removed	Gains
0	240	2.73	1.51	45 %	0.99–1.11
1	240	2.98	2.29	23 %	0.97–1.03
2	240	2.32	1.51	35 %	0.98–1.03
3	128	6.97	4.26	39 %	0.90–1.05
4	90	1.09	0.75	31 %	0.98–1.01

Table 19: Depth refinement before TSDF fusion on Jal Mahal’s two largest models: view completeness, median mesh depth error against the triangulated depth, triangles, and refinement time for all keyframes.

Model	Filter	Compl.	Depth err. (%)	Triangles	s
jal_mahal 0	none	0.950	1.59	674,622	0.00
jal_mahal 0	guided	0.959	1.75	741,762	0.45
jal_mahal 0	fgs	0.960	1.91	693,983	0.23
jal_mahal 1	none	0.830	0.59	735,962	0.00
jal_mahal 1	guided	0.719	0.56	683,599	0.43
jal_mahal 1	fgs	0.661	0.58	633,168	0.16

6 Limitations

Budget. On the warm engine 1 of the 1 sample videos are processed within the budget; the misses are fast FPV flights and clips under half a minute (§5.7). **Completeness.** About three quarters of what the keyframes saw is reconstructed; surfaces the single pass never faced cannot be, and the far field rests on a monocular prior. **No flight logs in the public samples.** The fifteen sample videos are YouTube re-encodes without GPS; georeferencing is evaluated in simulation and with synthetic GPS on real models, and runs on real data only when a DJI SRT, CSV, GPX or JSON log is supplied. Accuracy degrades with distance from the track (7–18 m at 300 m and beyond). **Edited footage.** Each pass of an edit becomes its own model. **Water and sky.** Reflections are reconstructed as mirrored geometry below the surface. **Dynamic objects** are not masked. **Live segments** cut a camera move at their boundary. **Licences.** Depth Anything V2 Large, the fast profile’s default prior, is CC-BY-NC-4.0; the Apache-2.0 Small model can replace it. **Compositional objects.** WorldSculpt [17] could turn the scene into per-object meshes given instance masks and boxes; we have not integrated it.

7 Conclusion

A single drone pass can be turned into a textured, georeferenced mesh and point cloud on one GPU within the problem statement’s time budget, provided the system chooses its frames by what they see rather than by the clock, checks that the motion contains parallax before trusting it, reuses the optical flow it already computed for poses and depth instead of matching features, and aligns to GPS with the constraint a straight line cannot provide. The code, configurations and the scripts that regenerate every number in this paper are in the repository.

References

- [1] National Technical Research Organisation. Single-pass drone video to accurate 3D model generation system. Smart India Hackathon 2026, Problem Statement 26158, 2026.
- [2] Zachary Teed and Jia Deng. RAFT: Recurrent all-pairs field transforms for optical flow. In *European Conference on Computer Vision (ECCV)*, 2020.
- [3] Linfei Pan, Dániel Baráth, Marc Pollefeys, and Johannes L. Schönberger. Global structure-from-motion revisited. In *European Conference on Computer Vision (ECCV)*, 2024. arXiv:2407.20219.
- [4] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything V2. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.09414.
- [5] Brian Curless and Marc Levoy. A volumetric method for building complex models from range images. In *Proceedings of SIGGRAPH*, pages 303–312, 1996.
- [6] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. Open3D: A modern library for 3D data processing, 2018. arXiv:1801.09847.
- [7] Harry (J.X.) C. Spirula studio: Cross-vendor 3D Gaussian splatting trainer. <https://github.com/harry7557558/spirula-studio>, 2026. GitHub user harry7557558; version 2026.9.24, commit d8e6a26, GPL-3.0.
- [8] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023. arXiv:2308.04079.
- [9] Igor Pavlovic, Thiemo Wandel, Anton Obukhov, Luca Bartolomei, Andrey Davydov, Fabio Tosi, Matteo Poggi, Sabine Süsstrunk, and Dengxin Dai. Marigold V2: Revisiting diffusion transformers for monocular depth estimation. *ACM Transactions on Graphics*, 45(6), 2026. doi: 10.1145/3842528. arXiv:2609.08084.
- [10] Philip H. S. Torr. Geometric motion segmentation and model selection. *Philosophical Transactions of the Royal Society A*, 356(1740):1321–1340, 1998.
- [11] Johannes L. Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [12] Vickie Ye, Ruilong Li, Justin Kerr, Matias Turkulainen, Brent Yi, Zhuoyang Pan, Otto Seiskari, Jianbo Ye, Jeffrey Hu, Matthew Tancik, and Angjoo Kanazawa. gsplat: An open-source library for Gaussian splatting. *Journal of Machine Learning Research*, 26(34):1–17, 2025. arXiv:2409.06765.
- [13] Raúl Mur-Artal, J. M. M. Montiel, and Juan D. Tardós. ORB-SLAM: A versatile and accurate monocular SLAM system. *IEEE Transactions on Robotics*, 31(5):1147–1163, 2015.
- [14] Narayanan Sundaram, Thomas Brox, and Kurt Keutzer. Dense point trajectories by GPU-accelerated large displacement optical flow. In *European Conference on Computer Vision (ECCV)*, 2010.
- [15] Bingxin Ke, Anton Obukhov, Shengyu Huang, Nando Metzger, Rodrigo Caye Daudt, and Konrad Schindler. Repurposing diffusion-based image generators for monocular depth estimation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. arXiv:2312.02145.
- [16] Qwen Team. Qwen-Image technical report, 2025. arXiv:2508.02324.

- [17] Muyao Niu, Jixuan He, Ruihan Yu, Lian Fu, Yonghao Yu, Zheng-Hui Huang, Yifan Zhan, Fengbo Lan, Yongtao Ge, Yinqiang Zheng, Kaipeng Zhang, and Zhixiang Wang. WorldSculpt: Generating compositional worlds from grounded videos, 2026. arXiv:2609.05416.
- [18] Richard I. Hartley. In defense of the eight-point algorithm. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 19(6):580–593, 1997.
- [19] Shinji Umeyama. Least-squares estimation of transformation parameters between two point patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(4):376–380, 1991.